Einführung in HTML

Einführung in HTML ... 3
Uniform Resource Identifiers (URIs) (RFC 3986) ... 3
Was ist HTML? ... 5
Grundstruktur einer HTML-Seite ... 7
Textformatierung ... 8
Schriftarten: ... 8
Überschriften ... 9
Zeilenumbrüche ... 9
Absatz ... 9
Vorformatierter Text ... 10
Seitengestaltung ... 11
Das RGB-Farbschema ... 11
Weitere HTML- Gestaltungselemente ... 12
Waagerechte Linie ... 12
Listen ... 12
Hypertext ... 13
Anker-Tag ... 14
Bilder ... 16
Bildformate ... 16
Aufgabe 1 ... 17
Image Maps (verweis- sensitive Grafiken) ... 18
Server- seitige Image Maps ... 18
Client- seitige Image Maps ... 19
Frames ... 20
Aufgabe 2 ... 21
Tabellen ... 22
Nested Tables ... 23
Aufgabe 3 ... 24

Einführung in HTML

Uniform Resource Identifiers (URIs) (RFC 3986)

Ist eine Folge von Zeichen die eine abstrakte oder physikalische Ressource identifiziert.

Eine URI kann weiter eingeteilt werden in einen locator URL oder einen name URN.

In Zukunft sollte URI anstatt von URL oder URN verwendet werden.

Beispiele:

https://www.google.com/calendar/render?tab=wc

http://fhwgt.de/

https://www.lsf.hs-weingarten.de/qisserver/rds?state=user&type=0

mailto:John.Doe@example.com

news:comp.infosystems.www.servers.unix

tel:+1-816-555-1212

telnet://192.0.2.16:80/

urn:oasis:names:specification:docbook:dtd:xml:4.1.2

URN: Uniform Resource Name
(RFC 2141)

Sollen als dauerhafte vom Ort unabhängige Ressource Identifiers verwendet werden.

URN syntax (phrases enclosed in quotes are REQUIRED):

<URN> ::= "urn:" <NID> ":" <NSS>
<NID> Namespace Identifier

<NSS> Namespace specific String

URL: Uniform Resource Locator

Identifiziert eine Rssource über ihren Speicherort und über den zu verwendenden Zugriffsmechanismus

Zukünftige Spezifikationen sollten statt der Begriffe URL oder URN den Begriff URI verwwenden.

Ein URI hat den folgenden allgemeinen Aufbau:

```
URI         = scheme ":" hier-part [ "?" query ] [ "#" fragment ]

    hier-part   = "//" authority path-abempty
                / path-absolute
                / path-rootless
                / path-noscheme
                / path-empty

authority   = [ userinfo "@" ] host [ ":" port ]
```

Beispiel:

http://www.fh-weingarten.de:8042/fbe/intro/welcome.html?name=xyz#marke1

Innerhalb von HTML-Dokumenten können Verweise auf andere Dokumente (Ressourcen) angegeben werden. Diese Verweise werden in Form von URIs angegeben. Ein solcher Verweis wird auch (Hyper-) Link genannt.

Ein URI sollte ursprünglich in eine der beiden Klassen URL oder URN eingeteilt werden. URI ist ein Überbegriff für URL und URN.

Ein Uniform Resource Name (URN) soll eine Quelle eindeutig referenzieren, die nicht durch ein Internetprotokoll abgerufen werden kann.

Uniform Resource Locators (URLs) waren ursprünglich die einzige Art von URIs daher wird der Begriff URL oft gleichbedeutend mit URI verwendet.

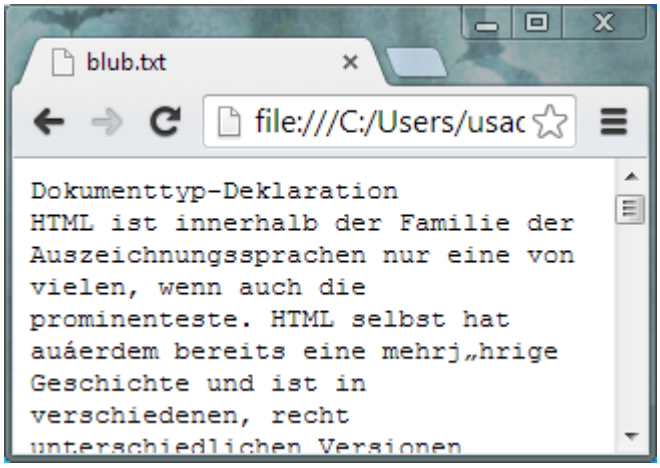
Was ist HTML?

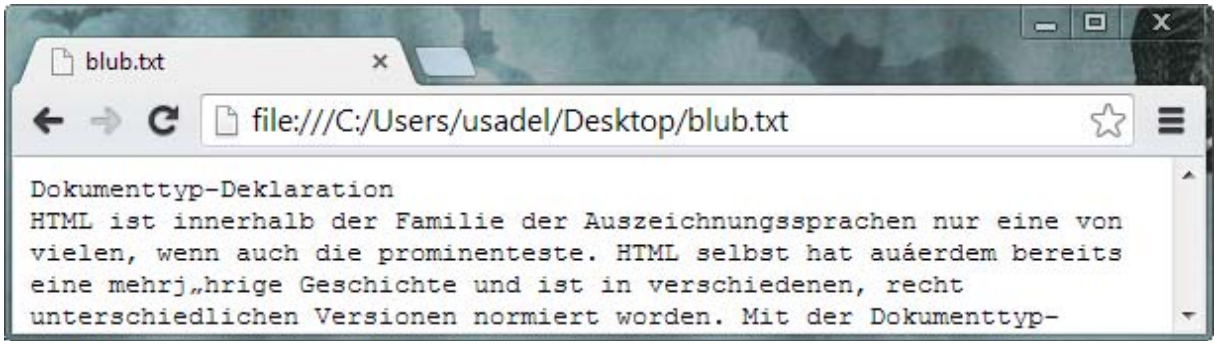
HTML steht für **H**yper**t**ext **M**arkup **L**anguage. Es ist eine Sprache in der WWW-Seiten beschrieben werden können.

Grundgedanke des Web-Dienstes;

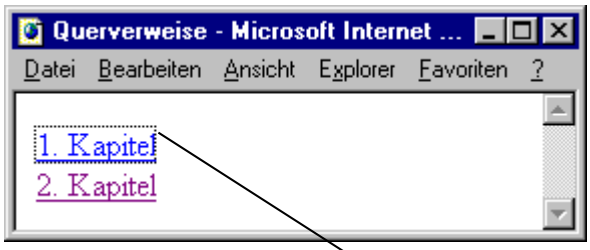
- Wissenschaftler wollten Forschungsergebnisse einfach abrufbar zur Verfügung stellen.

- Der Text passt sich dem Browserfenster an.





- Ein Anwender kann durch einfaches Anklicken Querverweisen folgen

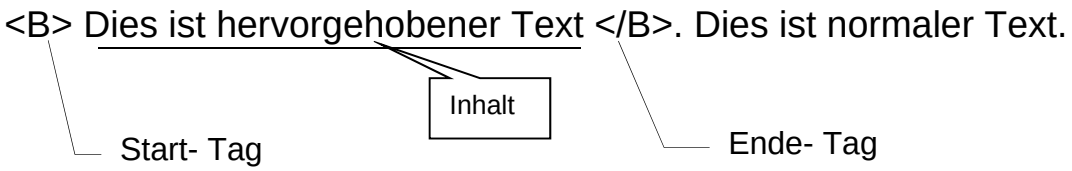



```
<html>
 <head>
   <title> Querverweise</title>
 </head>
 <body bgcolor =white>
  <basefont size=2>
  <A HREF="www.fh-weingarten.de/dir1/index.html#Kap1">1. Kapitel</A><BR>
  <A HREF="www.fh-weingarten.de/dir1/index.html#Kap2">2. Kapitel</A><BR>
 </body>
</html>
```

Mit HTML kann nicht nur Text dargestellt werden, sondern es können u.a. auch Bilder, Formulare, Audiodateien und Videoclips in ein HTML-Dokument eingebunden werden.

Text besteht bei Textverarbeitungsprogrammen aus sichtbarem Text und versteckten Formatierungskodes. Die Formatierungskodes sind dem Anwender normalerweise verborgen.

Auch HTML-Dokumente bestehen aus diesen Bausteinen. Die Formatierungskodes heißen hier Tags. Die Tags werden durch spitze Klammern gekennzeichnet. Eine Auszeichnung (Markup) besteht aus einem Start- Tag, dem Inhalt und einem Ende- Tag.

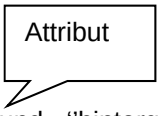


Ein Tag seinerseits besteht aus Elementen und Attributen. Im obigen Beispiel wird kein Attribut verwendet. Es ist unerheblich ob die Elemente oder Attribute groß oder klein geschrieben werden. Es gibt Tags die, wie im obigen Beispiel, aus einem Start- und einem Ende- Tag bestehen. Es gibt allerdings auch einige Tags, die nur aus einem Start- Tag bestehen. Ein Beispiel hierfür ist das
-Tag. (BR steht für Linebreak und fügt einen Zeilenumbruch ein.

Der Browser interpretiert die Tags und formatiert den zwischen Start- und Ende- Tag eingeschlossenen Text gemäß der Formatierungsanweisungen der Tags. Zeilenumbrüche, die im Text enthalten sind, werden vom Browser in der Regel ignoriert. Er erzeugt Zeilenumbrüche, wenn das nächste darzustellende Wort nicht mehr in der aktuellen Zeile des Browserfenster angezeigt werden kann.

Bei vielen Tags kann man zusätzlich noch Attribute angeben. Diese legen die Eigenschaften des zugehörigen Tag fest. Zwischen Tag und Attribut muß ein Leerzeichen eingefügt werden.

Ein Beispiel:



<BODY background =‘’hintergrundbild.gif“> .....</BODY>

Der Wert nach dem Gleichheitszeichen wird dem Attribut zugewiesen. Bei diesem Wert, der in der Regel in Anführungszeichen eingeschlossen wird, kann es sich um eine Zahl, einen Dateinamen, eine URL oder einen symbolischen Namen für einen Attributwert handeln. Der allgemeine Aufbau eines Markups stellt sich wie folgt dar:

<Element- Name attribut = "Attributwert"> Inhalt </Element- Name>

Beispiele:

```
<H1 align = "center"> Dies ist eine &Uuml;berschrift </H1>

Dies ist Text <BR>mit einem Zeilenumbruch .

Nach diesem Text wird eine Grafik eingefügt:
<IMG src = "bild.gif">

Elemente können grundsätzlich mehrere Attribute besitzen:
<TABLE align="center" width="75%" height="200">.....</TABLE>
```

Kommentare

Text, der nicht durch den Browser dargestellt werden soll, wird in einem HTML-Dokument als Kommentar angegeben.

<!-- Kommentar: Dieser Text wird nicht angezeigt -->

Anfang des Kommentars

Ende des Kommentars

Sonderzeichen (Entities)

Die spitzen Klammern sind für die Kennzeichnung der Tags reserviert. Daher dürfen sie im normalen Text nicht auftauchen. Ein weiteres Problem ergibt sich mit den nationalen Sonderzeichen(ä, ö, ü, ß....). Diese gerade genannten Zeichen kann ein Browser durch die Verwendung von Symbolkürzeln (Entities) darstellen. Eine kleine Auswahl:

Sonder-zeichen	**Entity**
Ä	Ä
ä	ä
ß	ß
>	>
<	<

Die Entities werden durch die Zeichen & und ; begrenzt. Eine andere Art der Darstellung ist die Angabe des dezimalen ASCII- Kodes des Zeichens mit einem vorangestellten #.

ä wird in dieser Schreibweise wie folgt angegeben: ä

Grundstruktur einer HTML-Seite

Angabe der im HTML-Dokument verwendeten HTML-Version

Dokumentenkopf Dokument-Header

Dokumentenkörper Dokument-Body

```
<!DOCTYPE HTML PUBLIC "-//W3C//DTD HTML 3.2//EN">
<html>
  <head>
        <title> Text des Browsertitelfensters</title>
  </head>

  <body>
     <b>Dieser Text erscheint fett.</b>
     <i>Dieser Text erscheint kursiv.</i>
   </body>
</html>
```

Diese Seite wird im Browser wie folgt dargestellt:

Im Dokumentenkopf befinden sich Informationen über das Dokument. Z.B. der Titel des Dokumentes, der sich zwischen den Tags <TITLE> und </TITLE> befindet.

Ein weiteres Tag, welches an dieser Stelle stehen muß -falls man es verwendet- ist das <META>Tag. Mit Hilfe des META- Tag lassen sich Informationen über das Dokument angeben. Diese Informationen können von Suchmaschinen ausgewertet werden.

<META NAME=‘’Author“ CONTENT=‘’ Simmel“>

<META NAME=‘’keywords“ CONTENT=‘’ stichworte“>

<META NAME=‘’GENERATOR“ CONTENT =‘’Mozilla/3.0Gold (Win 95; I) [Netscape] >

<META HTTP-EQUIV=‘’Expires“ CONTENT=‘’Tue, 24 Dec 1996 06:00:00 GMT“>

<META HTTP-EQUIV=‘’refresh“ CONTENT=‘’n; neuer URL“>

Der eigentliche Inhalt eines Dokumentes befindet sich zwischen den Tags <BODY> und </BODY>.

Textformatierung

einige Schriftarten (Stile)

<CITE>	Zitat	</CITE>
<CODE>	Code	</CODE>
<DFN>	Definition	</DFN>
<EM>	hervorgehoben	</EM>
<KBD>	Tastatureingabe	</KBD>
<SAMP>	Beispiel	</SAMP>
<STRONG>	stark	</STRONG>
<VAR>	Variable	</VAR>
<B>	Fettdruck (bold)	</B>
<I>	kursive Schrift (italic)	</I>
<STRIKE>	durchgestrichen	</STRIKE>
<TT>	diktengleich	</TT>

Schriftarten:
Zitat
Code
Definition
hervorgehoben
Tastatureingabe
Beispiel
stark
Variable
Fettdruck (bold)
kursive Schrift (italic)
~~durchgestrichen~~
diktengleich

```
<html>
  <head>
    <title> Schriftarten</title>
  </head>
  <body>
   <b>Dieser Text erscheint fett.</b>  <br>
   <i>Dieser Text erscheint kursiv.</i><br>
    <u>unterstrichen </u><br>
    <small> small</small><br>
     normal<sub>tiefer</sub>normal<br>
     normal<sup>h&ouml;her</sup>normal<br>
     normal<s>durchgestrichen</s><br>
  </body>
</html>
```

Überschriften

Die Schriftgröße für Überschriften wird durch folgende Tags gesteuert:

<h1> </h1> Größte Überschrift
<h2> </h2>
.

<h6> </h6>Kleinste Überschrift

Zeilenumbrüche

 	Es wird ein Zeilenvorschub eingefügt.
<nobr> </nobr>	Beim Text zwischen diesen Tags werden Zeilenumbrüche vom Browser übernommen
wort1 wort2	Der automatische Umbruch zwischen wort1 und wort2 wird unterbunden

Absatz

<p> Der Beginn eines neues Absatzes wird so markiert. Es werden ein Zeilenumbruch und eine Leerzeile eingefügt.

Ein Absatz geht bis zum Beginn eines neuen Absatzes.

Es sind auch Attribute möglich:

<P ALIGN=LEFT> ...	</P> Der Absatz wird linksbündig ausgerichtet.
<P ALIGN=CENTER> ...	</P>
<P ALIGN=RIGHT> ...	</P>
<P ALIGN=JUSTIFY> ...	</P> Text im Blocksatz (wird noch nicht unterstützt)

Vorformatierter Text

Eine Möglichkeit formatierten Text in ein HTML- Dokument zu übernehmen ist die Verwendung des <PRE>-Tag. Damit bleiben auch Texttabellen erhalten. Nach dem <PRE>-Tag übernimmt der Browser die Leerzeichen, Tabulatoren und Zeilenumbrüche des HTML-Dokumentes.

Beispiel:

```
<PRE>
+-------------------------------------------------------------------------------+
|       | Montag      | Dienstag    | Mittwoch    | Donnerstag  | Freitag     |
|-------------------------------------------------------------------------------|
|  8.00 |             | MCOM2 JOBK  |             | MCOMP JOBK  | MATHI ERT   |
|       |             | T117        |             | H216        | K103        |
|-------------------------------------------------------------------------------|
|  9.45 | RT1   ADER  | RT1   ADER  | MATHI ERT   | MCOMP JOBK  | MATHI ERT   |
|       | N042        | N042        | K103        | H216        | K103        |
|-------------------------------------------------------------------------------|
| 11.45 |             |             | RT1   ADER  | MSOFP KOCH  | SYSOP KEL   |
|       |             |             | N042        | T111        | K104        |
|-------------------------------------------------------------------------------|
| 14.15 | OOPR  GAM   | ETHIK WIL   |             | DABS  HUL   | DABS  HUL   |
|       | T013  I028  | H142        |             | K103  K104  | K103        |
|-------------------------------------------------------------------------------|
| 16.00 | OOPR  GAM   |             |             | RT1   ADER  |             |
|       | T013  I028  |             |             | N042        |             |
|-------------------------------------------------------------------------------|
| 17.45 |             |             |             |             |             |
|       |             |             |             |             |             |
|-------------------------------------------------------------------------------|
</PRE>
```

Seitengestaltung

<body >...</body> Umklammert den Körper eines HTML-Dokumentes.

Attribute des body-tag:

background = '' ...“	URL eines Bildes, das als Seitenhintergrund dargestellt wird.
bgcolor ="#66CC00“	Farbe des Seitenhintergrundes *
text ="...“	Farbe des Seitentextes*
link="...“	Farbe des Linktextes von noch nicht abgerufenen Verbindungen*
alink="...“	Farbe von aktiven Verbindungen*
vlink="...“	Farbe des Linktextes von bereits abgerufenen Verbindungen*

*(Angabe im RGB-Farbschema)

Beispiel:

```
<body bgcolor="#ffffff text="#ff0000" link="#0000ff">
    Hier befindet sich der Nutzinhalt, der wie in den Attributen angegeben
    formatiert wird.
    Link
</body>
```

Das RGB-Farbschema

Bei der Darstellung von Farben im Computer setzt sich jede Farbe aus den drei Grundfarben Rot, Grün und Blau zusammen. Jeder dieser Farbanteile wird durch eine Zahl zwischen 0 und 255 dargestellt. Die Angabe erfolgt mit Hexadezimalzahlen.

bgcolor ="#66CC00"

Angabe für den Rotanteil

Es gibt ferner vordefinierte Farbnamen. Diese lassen sich anstelle der Hexadezimalzahl verwenden.

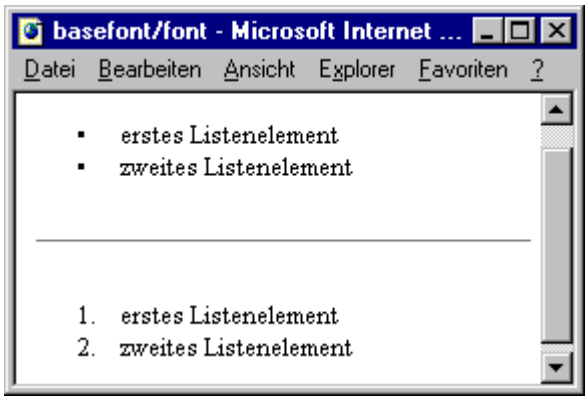
z.B.: **bgcolor=Aqua, Navy, Silver...**

Weitere HTML- Gestaltungselemente

Waagerechte Linie

<hr>	
<HR SIZE=number>	Dicke der Trennlinie
<HR WIDTH=number|percent>	Ohne eine Width-Angabe wird die Trennlinie über die gesamte Seitenbreite gezogen.Mit WIDTH=number gibt man die Pixelanzahl der Trennlinie an
<HR ALIGN=left|right|center>	Hiermit läßt sich die Ausrichtung einer Trennlinie angeben
<HR NOSHADE>	Die Trennlinie wird als einfache Linie gezeichnet.

Listen




```
<basefont size=1>
 <ul>
   <li>erstes Listenelement
   <li>zweites Listenelement
  </ul>
<hr>
<ol>
   <li>erstes Listenelement
   <li> zweites Listenelement
</ol>
```

<OL> </OL> : geordnete Liste
<UL> </UL> : ungeordnete Liste
....
<LH> </LH> : Listenüberschrift

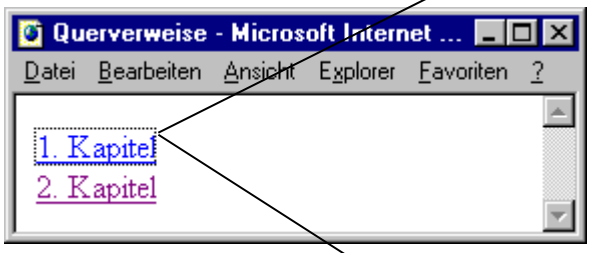
Attribute:

ALIGN	Legt die Ausrichtung fest (LEFT, RIGHT, CENTER JUSTIFY)
START	Numerierung beginnt nicht bei Null
TYPE	legt das Aufzählungszeichen fest (SQUARE, CIRCLE,DISC)

```
<OL  ALIGN= CENTER START=4 TYPE = SQUARE>
```

Hypertext

Auch in Lexika gibt es Querverweise (siehe auch ...). Im WWW gibt es „elektronische“ Verweise auf andere Dokumente oder auf Textstellen innerhalb desselben Dokumentes. Diese „elektronischen Verweise“ werden als (Hyper-)Links bezeichnet. Dem Benutzer wird die Information von welchem Ort und wie das Dokument geholt werden kann verborgen. Er bekommt nur einen besonders gekennzeichneten Querverweistext angeboten. Klickt er auf diesen Text, wird das Dokument von dem Ort und mit dem Verfahren geholt, wie es im elektronischen Verweis (dem Benutzer verborgen) angegeben ist.




```
<html>
 <head>
   <title> Querverweise</title>
 </head>
 <body bgcolor =white>
  <basefont size=2>
  <A HREF="www.fh-weingarten.de/dir1/index.html#Kap1">1. Kapitel</A><BR>
  <A HREF="www.fh-weingarten.de/dir1/index.html#Kap2">2. Kapitel</A><BR>
 </body>
</html>
```

Ein Hyperlink kann auf folgendes verweisen:

- eine andere Seite des gleichen WEB-Servers
- eine Seite eines anderen WEB-Servers (im lokalen Netz oder im Internet)
- eine Sprungmarke auf derselben Seite
- eine Sprungmarke auf einer anderen Seite
- eine FTP-, Gopher-, Telnet- oder Newsgroup-Adresse
- eine Email-Adresse

Anker-Tag

In einem HTML-Dokument legt man einen solchen Querverweis an, indem man das **Anker-Tag <a>...</a>** verwendet. Der Querverweistext wird in das Anker-Tag eingebettet, ebenso der „elektronische Verweis“. Dieser bleibt jedoch zunächst für den Benutzer verborgen.

Hinweis für den Browser um welche Art von Daten es sich handelt.

Name des WWW-servers

Name des Pfades

Name der Datei

< a href="http://www.fh-weingarten.de/fh/dir1/welcome.html"> Querverweistext </a>

der „elektronische“ Verweis (Link) ist ein **URL**

Dieser Text wird im Browser dargestellt und als anklickbar markiert

benannte Anker

Wenn man in einem Dokument an eine bestimmte Stelle verzweigen will, verwendet man das Anker-Tag mit dem Attribut NAME, um diese Stelle zu markieren.

Der Sprung an eine Marke im Text wird wie folgt ermöglicht:

<a href="#Sprungziel1"**>** Gehe zur Textpassage1**</a>**

Sprung im selben Dokument. Der Sprung aus einem anderen Dokument schließt URL mit ein.

...
...
...

<a NAME="Sprungziel1"**>** Beginn der Textpassage1**</a>**

URL

Ein URL(Uniform Resource Locator) bezeichnet, wo ein Dokument im Internet zu finden ist. Er ist wie folgt aufgebaut:

Protokoll://Domain-Name[:Port]/Pfad/Dokument

Protokoll:

Hier wird festgelegt, mit welchem Protokoll die Übertragung des Dokumentes oder der Daten gesteuert werden soll. Es sind u.a. die folgenden Protokolle möglich:

- http: Übertragung von WWW-Daten
- ftp: Übertragung von Dateien
- file: Übertragung von WWW-Daten vom eigenen Rechner

Domain-Name:

Dies ist der im Internet eindeutige Name des Rechners, der das gesuchte Dokument enthält. Ein Domain Name Server(DNS) verwaltet zu diesem Namen eine eindutige IP-Adresse.

Port:

Eine optionale Angabe. Sie bezeichnet Port unter dem der Server in das Netz „hört“. WWW-Server benutzen standardmäßig den Port 80.

Pfad:

Gibt den Pfad auf dem WWW-Server an, unter dem das angeforderte **Dokument** gespeichert ist.

Wenn in den Dokumenten immer die gesamte URL genannt werden müßte, wäre es mit großem Aufwand verbunden die Dokumente auf einen anderen Server oder in ein anderes Verzeichnis zu verschieben. Daher gibt es relative (oder partielle) URLs. Der Browser sendet jedoch immer eine vollständige URL, wenn er ein Dokument anfordert.

relativer URL

Aus einem relativen URL wird ein vollständiger URL auf folgende Weise ermittelt:
Beim relativen URL wird das äußerst links stehende Begrenzungszeichen ermittelt.
Dieses wird im gerade gültigen vollständigen URL gesucht.
Alles, was beim gerade gültigen URL links von diesem Begrenzungszeichen steht, bleibt erhalten, alles, was rechts davon steht, wird durch den relativen URL ersetzt.

Beispiel:

vollständiger URL: http://www.fh-ulm.de/pfad1/datei1.html#marke1

relative URLs:

1.) Das Protokoll bleibt gleich:

://www-fh-konstanz.de/test23/dat.html

2.) Protokoll, Server, Pfad und Dateiname bleiben gleich:

#marke2

Dies ist ein Sprung innerhalb eines Dokumentes

3.) Der gesamte Pfad ändert sich:

/pfad23/datei27.html

4.)Werden keine Begrenzungszeichen angegeben, ändert sich nur der Dateiname.

Bilder

<IMG SRC= ‘’bild.gif’’> fügt den Inhalt einer Bilddatei in die HTML-Seite ein. Mit Hilfe des Attributes ALT kann optionaler Text angegeben werden, der dann dargestellt wird, wenn die Bilddarstellung abgeschaltet ist.
<IMG SRC="triangle.gif" ALT="Warning:">

Das Ausrichten von Bildern erreicht man mit dem Attribut ALIGN. Left und right richten das Bild selbst aus, die anderen Werte des ALIGN- Attributes beziehen sich auf die Ausrichtung des das Bild umgebenden Textes relativ zum Bild.
<IMG SRC="triangle.gif" ALIGN= left|right|top|texttop|middle|absmiddle|
baseline|bottom|absbottom>

Mit dem BORDER-Attribut kann die Breite des Rahmens um ein Bild angegeben werden.(Der Vorgabewert ist 0, also wird kein Rahmen angezeigt.) Ist die Grafik als Link definiert wird standardmäßig ein Rahmen angezeigt. Dieser kann mit der Angabe BORDER=0 ausgeschaltet werden.
<IMG SRC="triangle.gif" BORDER=5>

HSPACE wird angewendet, um neben der Grafik eine Anzahl von Pixel freizulassen.

VSPACE wird angewendet, um über und unter der Grafik eine Anzahl von Pixeln freizulassen.
<IMG SRC="triangle.gif" HSPACE=10 VSPACE=5>

Die Größe des Bildes kann mit den beiden Attributen HEIGHT für die Höhe und WIDTH für die Breite verändert werden. Wird nur eines der beiden Attribute verändert, wird die Grafik proportional vergrößert oder verkleinert.
<IMG SRC="triangle.gif" WIDTH=100 HEIGHT=300>

Ein Bild kann auch als Hypertextreferenz verwendet werden:
<a href=“http://www.xxx.de/datei.html><IMG SRC= ‘’test.gif’’></a>

Bildformate

Im WWW sind die am häufigsten verwendeten Bildformate GIF und JPEG.

GIF (Graphics Interchange Format)

- Wurde vom CompuServe Information Service entwickelt.
- es gibt folgende wichtige Varianten :
  - GIF 87a : Die Bilder werden auf eine Größe von 20% bis 25% ihrer Originalgröße komprimiert. (LZW Algorithmus)
  - GIF 89a : Zusätzlich sind bei diesem Format transparente Hintergründe möglich. Nachteil deser GIF-Formate : Es werden nur 256 Farben benutzt.

- GIF24 : Unterstützt werden mehr als 16 Millionen Farben. Falls die Grafikkarte nur 256 Farben (24-Bit-Farben) unterstützt, müssen allerdings die 24-Bit Farben durch Dithering-Techniken reduziert werden.

JPEG (Joint Photographic Expert Group)

- Durch o.g. Gruppe entwickelt, um Standbilder zu komprimieren.
- Unterstützt 24-Bit-Farben
- Reduktion bis auf 20% möglich
- Ist verlustbehaftet (Bildinformationen gehen verloren)
- Ist gut geeignet für komplexe Bilder und digitalisierte Fotos.
- Nicht geeignet für einfache Bilder mit wenigen Farben.

Aufgabe 1

Aufgabe 1

Legen Sie eine HTML-Seite an!

1.) Verwenden Sie eine zentrierte waagerechte Linie mit einer Breite von 50% des Browser-Fensters.

2.)Geben Sie Ihrer Seite eine weiße(blaue) Hintergrundfarbe und dem Text eine graue Farbe.

3.)In dieser Seite soll eine geordnete und eine ungeordnete Liste vorkommen.

4.)Legen Sie einen Verweis an eine bestimmte Textstelle innerhalb dieses Dokumentes mit Hilfe eines benannten Ankers an.

5.)Legen Sie Links zu zwei weiteren Dokumenten an. Eines der Dokumente soll sich in einem anderen Pfad als das Ursprungsdokument befinden.

6.)Fügen Sie ein Bild in diese Seite ein.

Image Maps (verweis- sensitive Grafiken)

sind Grafiken, bei denen verschiedene Bereiche der Grafik auf unterschiedliche URLs verweisen. Klickt man diese Stellen (Hotzones) im Bild an, wird ein mit dieser URL verbundenes Dokument abgerufen und im Browser dargestellt. Man unterscheidet grundsätzlich Server- und Client- seitige Image Maps.

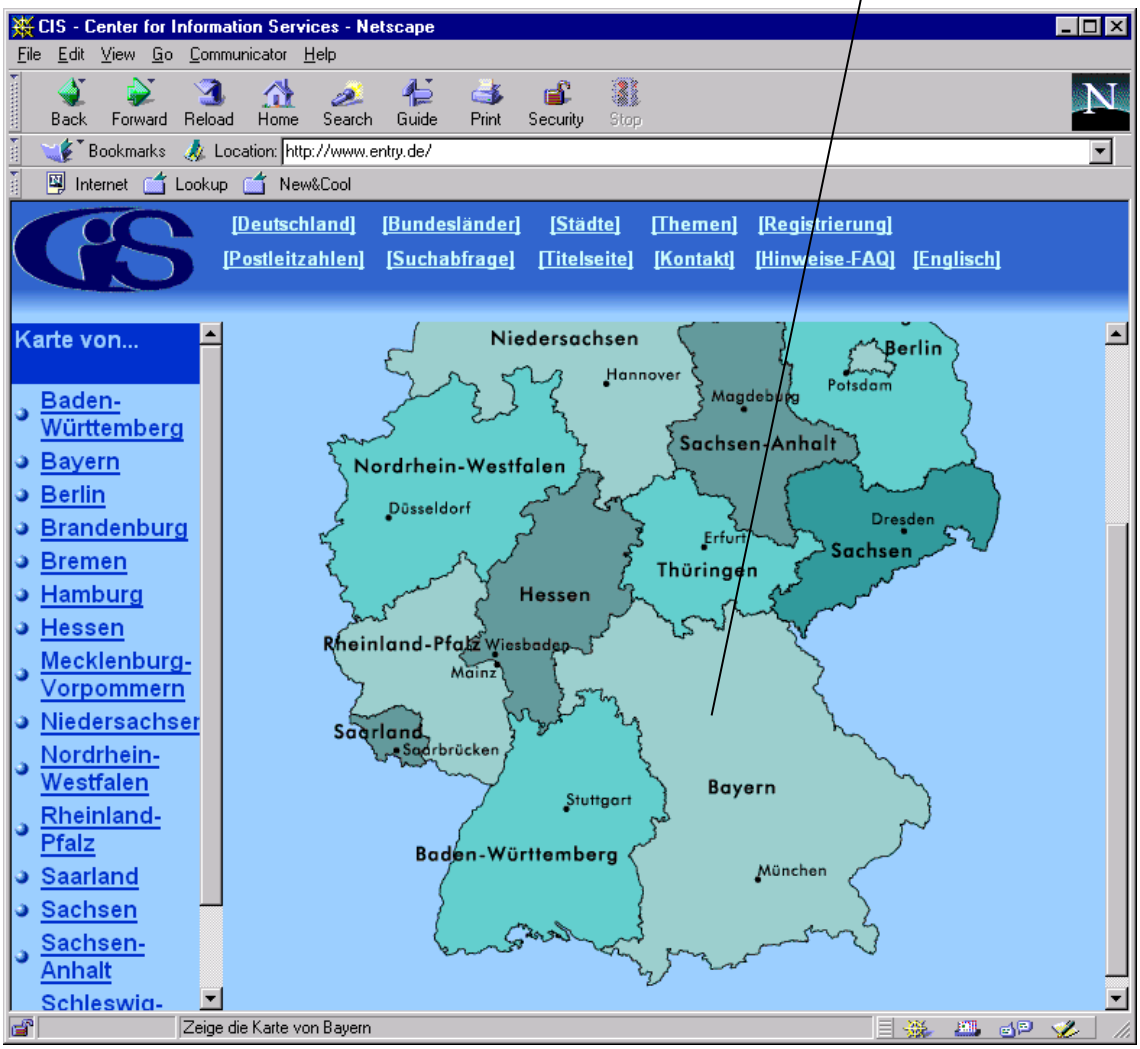
Server- seitige Image Maps

Bei diesen Maps werden die Koordinaten, die im Bild angeklickt werden, an den Server gesendet. Auf dem Server wird ein Programm gestartet, welches aus den Koordinaten die gewünschte URL ermittelt.
Im folgenden Beispiel befindet sich das Programm, welches aus den Koordinaten eine URL ermittelt, im Verzeichnis /cgi-bin/imagemap/ des Servers. Die Datei, die die Zuordnung der Koordinaten zu URLs enthält heißt hier karte.map.

Die Klickkoordinaten werden in der Form ?xzahlwert,yzahlwert an die im Link aufgeführte URL angehängt (beispielsweise ?23,16.)

Beispiel:

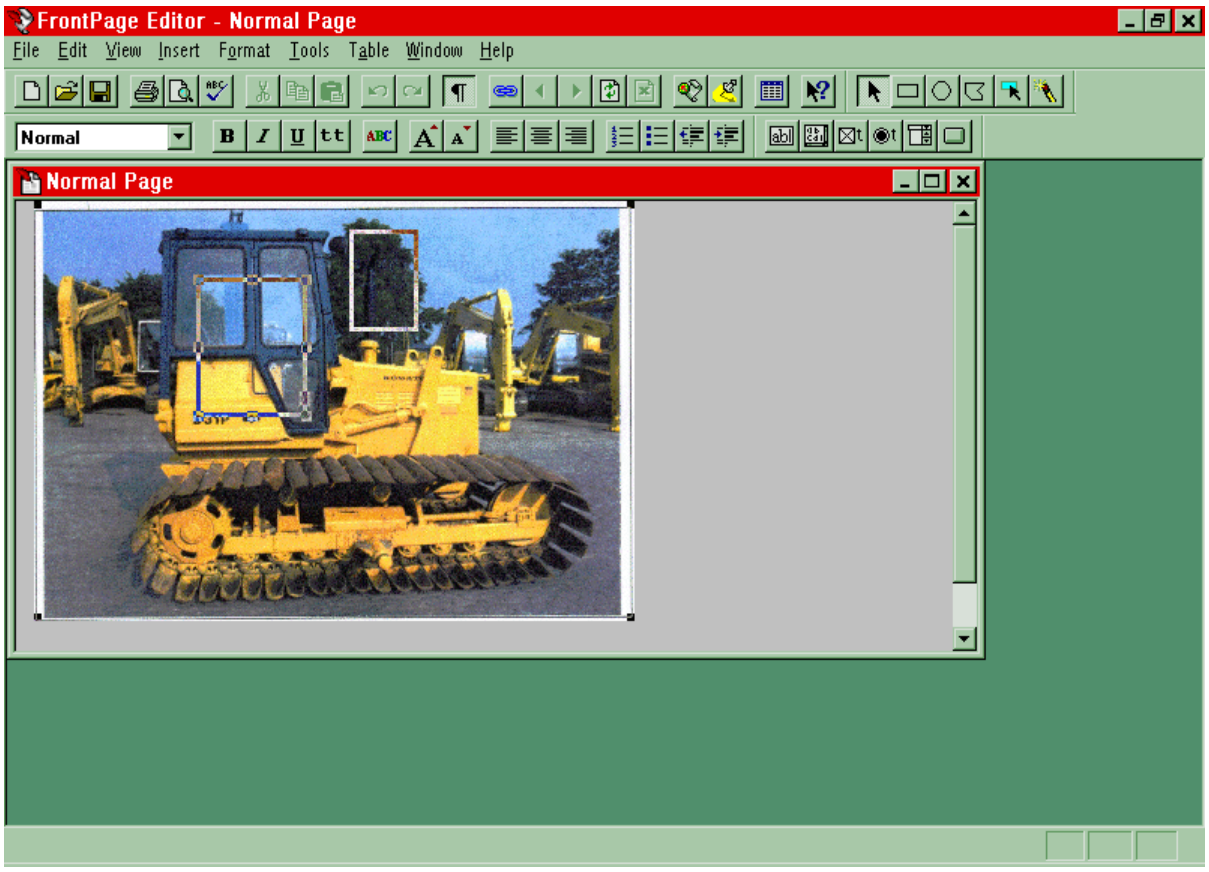
<A HREF= "/cgi-bin/imagemap/karte.map"> <IMG src="Ukarte.gif" ISMAP> </A>



Der Aufbau von Map- Dateien hängt vom jeweiligen CGI-Programm ab. Beim CERN-Server haben sie den folgenden Aufbau:
default URL
rectangle (left-x,top-y) (right-x,bottom-y) URL
circle (center-x, center-y) radius URL
polygon (x1,y1) (x2,y2) (x3,y3) ....(Xn,Yn) URL
point (x,y) URL

Client- seitige Image Maps

Mit dem Usemap-Attribut des IMG-Markups gibt man das <MAP>-Element an, welches die verweis- sensitiven Bereiche eines Bildes definiert. Die Definition dieser Bereiche befindet sich innerhalb der HTML-Seite, die auch das IMG-Element enthält. Der Client wertet ohne die Zuhilfenahme eines Servers die Klickkoordinaten aus.



Listing des zugehörigen HTML-Dokumentes :

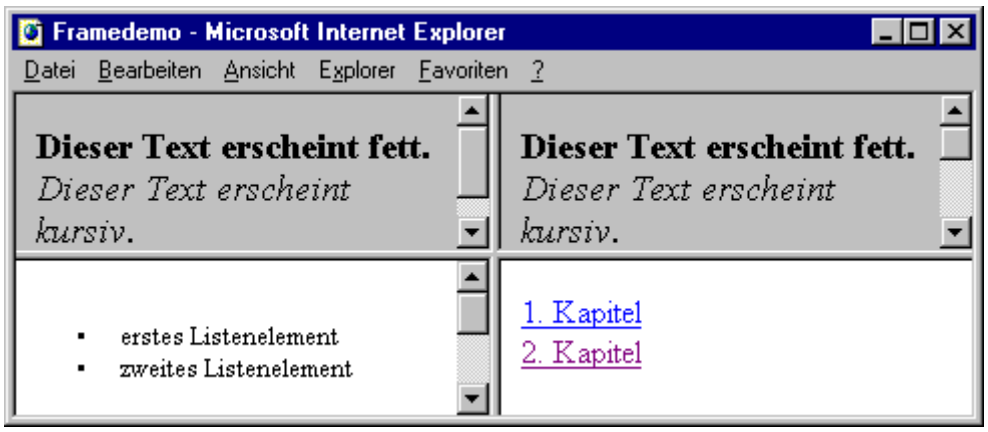
```
<HTML>
<HEAD>
        <TITLE>Normal Page</TITLE>
</HEAD>
<BODY>

        <MAP NAME="FrontPageMap" >
                <AREA SHAPE="RECT" COORDS="211, 20, 255, 86" HREF=   "a_baum.htm">
                <AREA SHAPE="RECT" COORDS="109, 52, 181, 143" HREF= "b_ant1.htm">
        </MAP>

         <IMG ALIGN="BOTTOM" SRC="k_lur400.gif" WIDTH="400"
          USEMAP="#FrontPageMap" HEIGHT="280" BORDER="0">

</BODY>
</HTML>
```

Ein Klicken innerhalb der Rechteckkoor-dinaten lädt diese Seite

In diesem Bild befinden sich zwei anklickbare Bereiche (Rechtecke). Diese Bereiche sind im <MAP>-Tag mit dem Namen FrontPageMap“ angegeben.

Frames

Mit Frames kann man ein Browserfenster in mehrere Bereiche unterteilen. Damit ist es beispielsweise möglich am Rand des Browserfensters eine Navigationsleiste zu plazieren. Das folgende Beispiel zeigt vier Frames.




```
<!DOCTYPE HTML PUBLIC "-//IETF//DTD HTML 3.0//EN//">
<HTML>
<HEAD>
<TITLE> Framedemo</TITLE>
</HEAD>

 <FRAMESET ROWS="50%,50%" COLS="50%,50%">
   <FRAME SRC="h1.html">
   <FRAME SRC="h2.html">
   <FRAME SRC="h4.html">
   <FRAME SRC="h5.html">
 </FRAMESET>
</HTML>
```

Im obigen Kasten ist das Frame-Dokument dargestellt, welches die darüberliegende Seite erzeugt. Es legt die Aufteilung in einzelne Frames fest und welche HTML-Seiten(h1.html bis h5.html) in die einzelnen Frames geladen werden. Ein Frame-Dokument enthält kein <BODY>-Tag. Die Aufteilung der Browserseite in Frames erfolgt mit dem <FRAMESET>-Tag.

<FRAMESET>

<FRAMESET COLS="30%,70%“> teilt das Browser-Fenster in zwei benachbarte (Spalten) Frames gleicher Höhe. Der linke Frame nimmt 30% der Browser-Fensterbreite ein. Der rechte Frame 70%.
Man hätte hier auch schreiben können:<FRAMESET COLS="30%,*">. Das Sternchen besagt, daß der verbleibende Rest der Breite für den zweiten Frame verwendet wird.

<FRAMESET ROWS="20%,20%,*“>legt drei (Zeilen) Frames übereinander an. Die ersten beiden Frames nehmen je 20% der Höhe des Browserfensters ein. Der dritte Frame erhält die verbleibende Höhe zugewiesen.

<FRAME>

Der Inhalt der einzelnen Frames wird mit Hilfe des <FRAME>-Tag festgelegt. <FRAME SRC="h1.html">. In der Reihenfolge wie man die HTML-Dokumente im Frame-Dokument angibt, werden sie von links nach rechts und von oben nach unten in den einzelnen Frames angezeigt.

Rollbalken ein-oder ausschalten

Mit dem Attribut SCROLLING des <FRAME>-Tag steuert man das Verhalten des Rollbalkens.

<FRAME SRC=''.....“ SCROLLING=NO>

Werte für die Attribute:

YES der Rollbalken wird auf jeden Fall eingeblendet
NO der Rollbalken wird nicht angezeigt
AUTO der Rollbalken wird dann angezeigt, wenn das anzuzeigende Dokument nicht vollständig im Frame angezeigt werden kann.

Frames, deren Größe konstant bleibt

Mit dem Attribut NORESIZE kann man verhindern, daß der Browsernutzer die Größe des Frame ändern kann.

<FRAME SRC=''.....“ NORESIZE>

Ein Dokument in einem bestimmten Frame ausgeben

Beispielsweise bei einer Navigationsleiste stellt sich die Aufgabe den Inhalt einer HTML-Seite in einem besonderen Fenster ausgeben zu müssen. Will man dies erreichen verwendet man im Frame-Dokument das Attribut NAME beim <FRAME>-Tag. Der Wert des NAME-Attributes gibt dem Frame einen Namen. Diesen Namen verwendet man, wenn man ein Dokument in diesem Frame anzeigen will.

<FRAME SRC=''h1.html“ NAME=''TEXT_Rahmen1“> Gibt diesem Frame den Namen TEXT_Rahmen1.

<A HREF=''hxx.html“ TARGET=''TEXT_Rahmen1“> Text anzeigen </A> :
Der Text des HTML-Dokumentes hxx.html wird beim Anklicken des Link im Frame mit dem Namen TEXT_Rahmen1 dargestellt.

Frames ohne Rahmen

finden z.B. Verwendung für eine Überschrift über einem scrollbaren Bereich

<FRAMSET ROWS=''20%,*“ FRAMEBORDER=NO>

Aufgabe 2

Aufgabe 2

1.)Legen Sie ein Frame-Dokument an, das ein Frameset mit 2 Zeilen und 2 Spalten definiert.

2.) Legen Sie ein Frame-Dokument an, das ein Frameset mit 2 Spalten definiert. Legen Sie im 1.Frame Links zu 2 verschiedenen HTML-Dokumenten an und geben Sie diese Dokumente im 2. Frame aus, wenn der betreffende Link angeklickt wird.

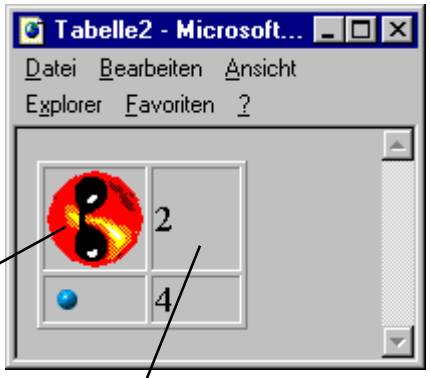
3.)Schalten Sie beim 1. Frame den Rollbalken ein.

Tabellen

Weil der Browser in der Regel Tabulatoren, Leerzeichen und Zeilenumbrüche ignoriert und weil es keine Positionsrahmen wie z.B. in Word gibt, sind die Tabellen das wichtigste Gestaltungsmittel in HTML-Dokumenten, um Text und Grafiken auf einer Seite auszurichten. Eine Tabelle besteht aus Tabellenzellen, die zeilenweise zu **Tabellenzeilen** zusammengefaßt werden.

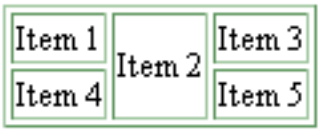
```
<TABLE BORDER=1>
        <TR> <TD>1</TD>  <TD>2</TD> </TR>
        <TR> <TD>3</TD>  <TD>4</TD> </TR>
</TABLE>
```

In die Tabellenzellen kann man Text und Grafiken einbinden. Im folgenden Beispiel sind zwei Grafiken eingebunden worden. Eines der Bilder ist ein animiertes Gif-Bild. Die Zelle mit der eingetragenen 4 ist mit Hilfe des WIDTH-Attributes auf eine Breite von 45 Pixeln gesetz worden. Damit paßt sich auch die darüberliegende Zelle auf diese Breite an.



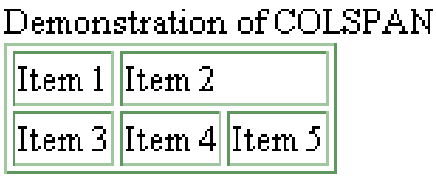

```
<TABLE BORDER=1>
        <TR> <TD><IMG SRC="ball.gif">      </TD>    <TD>                2  </TD> </TR>
        <TR> <TD><IMG SRC="ballblau.gif"></TD>    <TD WIDTH=45> 4  </TD> </TR>
</TABLE>
```

```
<TABLE BORDER>

<TR> <TD>Item 1</TD> <TD ROWSPAN=2>Item 2</TD><TD>Item 3</TD> </TR>
<TR><TD>Item 4</TD>                                  <TD>Item 5</TD> </TR>

</TABLE>
```

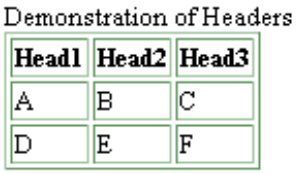


Demonstration of COLSPAN

```
<TABLE BORDER>

<TR><TD>Item 1</TD> <TD COLSPAN=2>     Item 2</TD></TR>
<TR><TD>Item 3</TD> <TD>Item 4</TD><TD>Item5</TD></TR>

</TABLE>
```



Demonstration of Headers




```
<TABLE BORDER>
<TR><TH>Head1</TH> <TH>Head2</TH> <TH>Head3</TH></TR>
<TR><TD>A</TD>      <TD>B</TD>      <TD>C</TD></TR>
<TR><TD>D</TD>      <TD>E</TD>      <TD>F</TD></TR>
</TABLE>
```

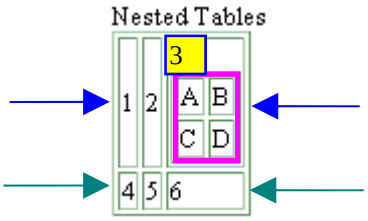
Sample table using all of the above

		Average	
		Height	Weight
Gender	Males	1.9	0.003
	Females	1.7	0.002

```
<TABLE BORDER>

<TR>
[illegible]
[illegible]</TR>
<TR>                                    <TH>Height</TH><TH>Weight</TH> </TR>
<TR><TH ROWSPAN=2>Gender</TH><TH>Males    </TH><TD>1.9</TD><TD>0.003</TD></TR>
<TR>                                <TH>Females </TH><TD>1.7</TD><TD>0.002</TD></TR>

</TABLE>
```

Nested Tables

```
<TABLE BORDER>
    <TR> <TD>1</TD><TD>2</TD><TD> 3    <TABLE BORDER>
                                           <TR><TD>A</TD><TD>B</TD></TR>
                                           <TR><TD>C</TD><TD>D</TD></TR>
                                       </TABLE>
                              </TD>
    </TR>
    <TR> <TD>4</TD><TD>5</TD><TD>6</TD></TR>
</TABLE>
```



Tag	Attribut	
<table > </table>		markiert den Anfang bzw. das Ende einer Tabelle
	border	fügt einen Rand um die Tabelle hinzu
	cellspacing	Platz, der zwischen den Zellen eingefügt wird
	cellpadding	Platz, der zwischen Zellenrand und Zelleninhalt eingefügt wird
	width	Angabe der Tabellenbreite in Pixeln oder in %
<caption>		Überschrift außerhalb einer Tabelle
<tr > </tr>		markiert den Anfang bzw. das Ende einer Tabellenzeile
<td > </td>		schließt Zellendaten ein
<th > </th>		schließt Zellendaten ein, diese werden fett gedruckt
	colspan	ändert die Anzahl der Spalten, die eine Zelle umfaßt
	rowspan	ändert die Anzahl der Zeilen, die eine Zelle umfaßt
	align	Ausrichtung des Zellinhaltes in horizontaler Richtung mögliche Werte : left/right/center
	valign	Ausrichtung des Zellinhaltes in vertikaler Richtung mögliche Werte: top/middle/bottom
	nowrap	Text in der Zelle wird nicht umgebrochen

Aufgabe 3

Aufgabe 3

Legen Sie eine HTML-Seite an!

1.) Legen Sie eine Tabelle mit 3 Zeilen und 3 Spalten an. Fügen Sie in die Zellen Bilder und Text ein.

2.) Stellen Sie Text neben einem Bild dar.

3.) Erstellen Sie mit Hilfe einer Tabelle dreispaltigen Text.

4.) Erstellen Sie eine Tabelle mit 2 Spalten und 2Zeilen und fügen Sie in die Zelle der ersten Spalte und ersten Zeile eine weitere 2X2 Tabelle ein.

5.) Ersetzen Sie im HTML-Code der 2.Tabelle auf Seite 18 die folgenden Zeichen

so, daß die Tabelle wie angegeben im Browser angezeigt wird.

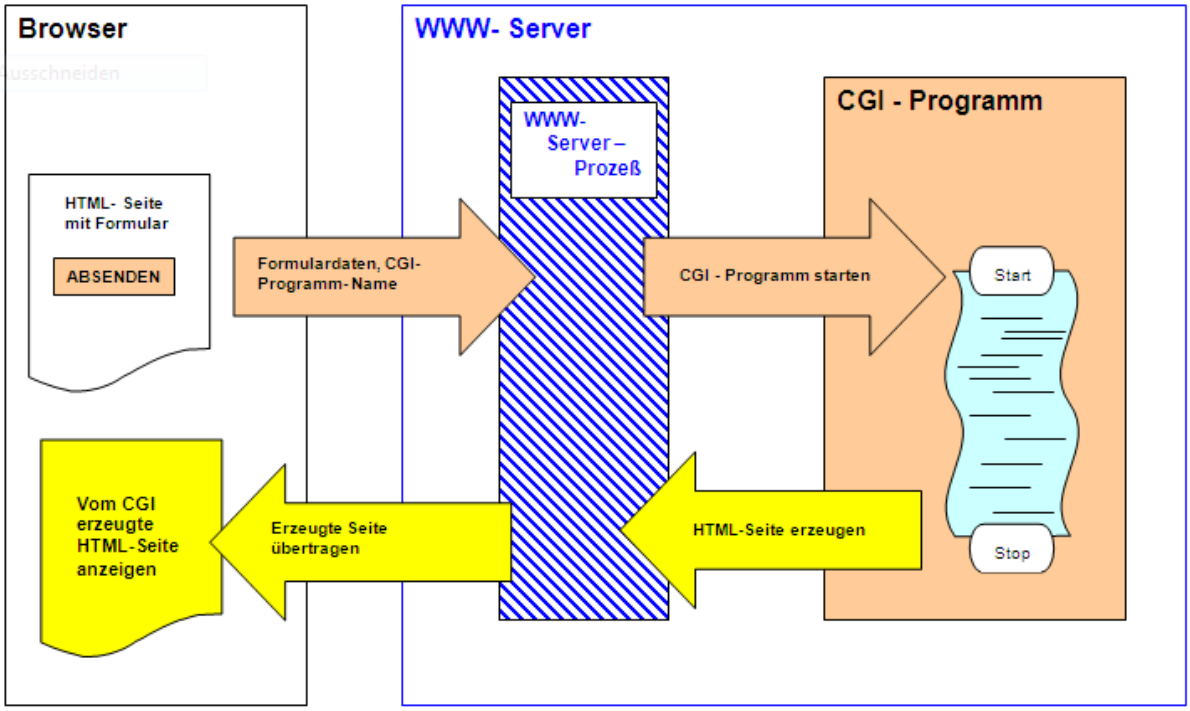
Einführung in HTML Teil 2

Formulare 27
CGI 27
Texteingabefelder 28
Listen 30
Aufgabe 4 31
CGI (Common Gateway Interface) 32
Übersicht der CGI-Umgebungsvariablen 39
Aufgabe 5 40

Formulare

Mit Hilfe von Formularen kann ein Besucher einer Web-Seite Eingaben vornehmen, die dann an den Web-Server gesendet werden, von dem die HTML-Seite mit dem eingebetteten Formular bezogen wurde. Das erfordert auf der Serverseite ein Programm, welches diese Daten entgegennimmt (z.B. ein CGI- Script). CGI steht dabei für Common Gateway Interface.

CGI



Zum Test der Formulare der folgenden Seiten liegen auf dem Webserver 141.69.142.10 die folgenden Dateien:

/var/www/ Aufrufpfad: 141.69.142.10/

form1.html
f_auswahlen1.html
f_radio.html
f_checkbox.html
f_text.html

/usr/lib/cgi-bin/ Aufrufpfad: 141.69.142.10/cgi-bin/

ps2
ls2
getvars
hello

In den folgenden Beispielen inter4.hs-weingarten.de/cgi-bin/query.pl durch 141.69.142.10/cgi-bin/getvars

Texteingabefelder

f_text.html

Lieber Nutzer,
bitte geben Sie Ihre E-Mail-Adresse ein.

Ihre E-Mail-Adresse lautet: Daten absenden

Tragen
Sie in diesem Feld Ihren Kommntar ein

Daten absenden

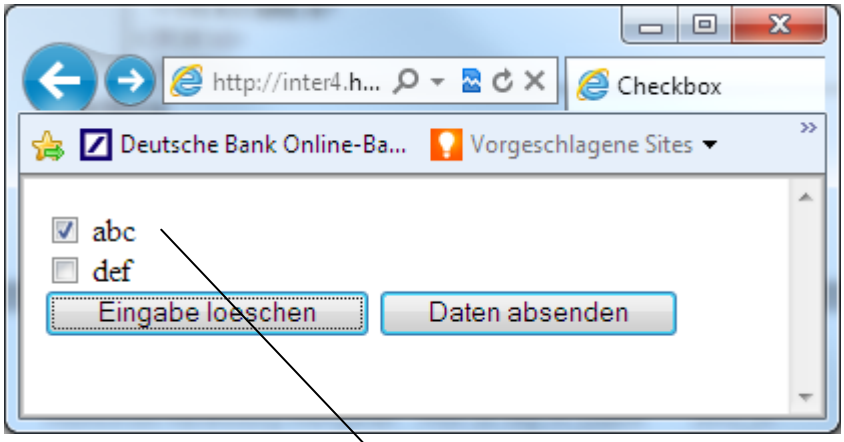
```
<HEAD>
<TITLE> Einzeilige Eingabe</TITLE>
</HEAD>

<BODY>
Lieber Nutzer,<br>
bitte geben Sie Ihre E-Mail-Adresse ein.<br><br>
<FORM METHOD=GET ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">
   Ihre E-Mail-Adresse lautet:
            <INPUT NAME="e_mail_address" TYPE=TEXT SIZE=30 MAXLENGTH=50>
<INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>
<hr>
<FORM METHOD=POST ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">

  <TEXTAREA NAME="Kommentar" ROWS=3 COLS=40>Tragen
Sie in diesem Feld Ihren Kommntar ein
   </TEXTAREA>
<INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>
</BODY>
```

Wenn man als Type PASSWORD angibt, wird der eingegebene Text nicht auf dem Bildschirm dargestellt

Checkbox

Dem Attribut VALUE kann ein Wert übergeben werden. Dieser wird an das CGI-SCript gesendet. Wurde kein Wert für Value angegeben, wird der Wert ON übersendet falls die Checkbox angewählt ist.

f_checkbox.html

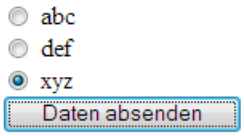



```
<HEAD> <TITLE> Checkbox</TITLE> </HEAD>
<BODY>
<FORM METHOD=POST ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">
  <INPUT NAME="cb1" TYPE=CHECKBOX CHECKED> abc<BR>
  <INPUT NAME="cb2" TYPE=CHECKBOX > def<BR>
 <INPUT TYPE=RESET VALUE="Eingabe loeschen">
 <INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>
</BODY>
```

Radiobutton

Radiobuttons lassen nur eine Auswahl zu.

f_radio.html




```
<HEAD> <TITLE>Radiobutton</TITLE></HEAD>
<BODY>
<FORM METHOD=POST ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">
  <INPUT NAME="rb" TYPE=RADIO CHECKED VALUE="abc"> abc<BR>
  <INPUT NAME="rb" TYPE=RADIO VALUE="def" > def<BR>
  <INPUT NAME="rb" TYPE=RADIO VALUE="3_rb" > xyz<BR>
  <INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>
</BODY>
```

Listen

```
<HEAD> <TITLE>List</TITLE></HEAD>
<BODY>

Einfachauswahl
<FORM METHOD=GET
   ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">
  <SELECT NAME="Standort" SIZE=1>
  <OPTION VALUE="ger" SELECTED> Deutschland
  <OPTION VALUE="uk" > Great Britain
  <OPTION VALUE="usa" > USA
  </SELECT>
  <INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>

Mehrfachauswahl
<FORM METHOD=GET
    ACTION="http://inter4.hs-weingarten.de/cgi-bin/query.pl">
  <SELECT NAME="Standort1" MULTIPLE>
  <OPTION VALUE="ger" SELECTED> Deutschland
  <OPTION VALUE="uk" > Great Britain
  <OPTION VALUE="usa" > USA
  </SELECT>
<INPUT TYPE=RESET VALUE="Eingabe loeschen">
<INPUT TYPE=SUBMIT VALUE="Daten absenden">
</FORM>
</BODY>
```

Einfachauswahl

Great Britain Daten absenden

Mehrfachauswahl

Deutschland
Great Britain
USA

Eingabe loeschen Daten absenden

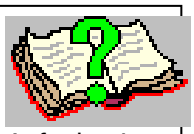
Formulardaten an den Server übertragen

Zu einem Formular gehört ein Mechanismus mit dem man die Daten des Formulares an den Server senden kann, von dem das Formular-Dokument bezogen wurde. Dazu wird durch das Attribut TYPE=SUBMIT des <INPUT>-Tag ein Button erzeugt. Wenn man diese Schaltfläche drückt, nimmt der Browser mit dem Server Verbindung auf und sendet die Formulardaten an den Server.

f_auswahlen1.html

Formulardaten löschen

<INPUT TYPE=RESET VALUE="Eingabe löschen"> erzeugt eine Schaltfläche mit der man die Eingaben eines Formulares verwerfen kann.



Aufgabe 4

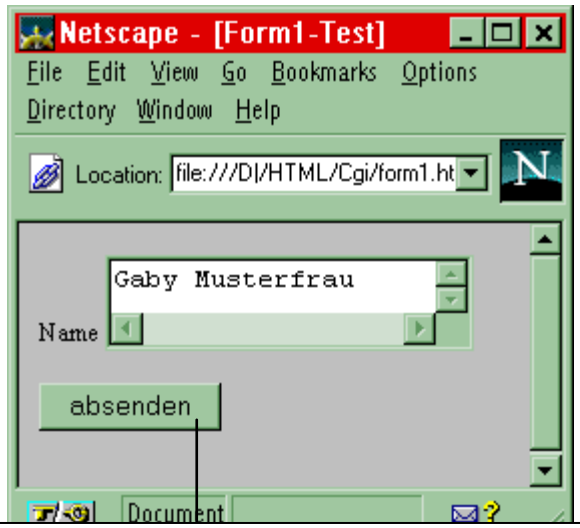
Aufgabe 4

Legen Sie eine HTML-Seite an!

1.) Erzeugen Sie für diese Seite ein Formular, welches ein einzeiliges und ein mehrzeiliges Eingabefeld sowie drei Checkboxen, 4 Radiobuttons, eine Einfachauswahl, eine Mehrfachauswahl und einen Submit- sowie Reset-Button enthält. Richten Sie diese Elemente mit Hilfe einer Tabelle aus.

Zum Testen können Sie die Daten an das CGI- Script "query.pl" des www-Servers "inter4.hs-weingarten.de" senden.

CGI (Common Gateway Interface)



Die Formulardaten des nebenstehenden Formulars werden mit dem Drücken des „absenden “-Buttons an den Server gesendet. Dieser nimmt die Daten entgegen. Dann startet der Server das betreffende CGI- Programm und liefert die Formulardaten an das CGI- Programm aus.
Die Schnittstelle, über die der Server mit dem CGI-Programm kommuniziert, ist das CGI.

Nutzung des CGI
1.)Der Browser fordert eine URL an, die auf eine Skriptdatei auf dem Server verweist. In diesem Fall : **query.pl**
Die Formulardaten werden ebenfalls an den Server gesendet.

Das Drücken dieser Taste löst das Senden der URL und der Formulardaten aus.

```
<HTML>
 <HEAD>
    <TITLE>Form1-Test</TITLE>
 </HEAD>
 <BODY>
    <FORM ACTION="http://inter4.hs-weingarten./cgi-bin/query.pl" METHOD="POST">
            Name <TEXTAREA NAME= "Question_name"></TEXTAREA>   <p>
             <INPUT TYPE="submit"   VALUE="absenden">
   </FORM>
 </BODY>
</HTML>
```

2.)Auf dem Server (inter4.hs-weingarten.de) wird das Skript **query.pl** ausgeführt.

Dieses Skript befindet sich beim Server im Unterverzeichnis /cgi-bin/. Der Server veranlaßt, daß der zugehörige Interpreter das CGI- Skript bearbeitet.

Infolge der Angabe der Methode POST im aufrufenden Formular werden dem CGI-Programm die Formulardaten über stdin übergeben. Das CGI- Programm (in diesem Fall ein Skript) verwendet für die Ausgabe stdout. Am Anfang einer Skriptdatei muss der Pfad des Programms angeben werden, welches das CGI-Skript interpretiert : **#!/usr/bin/perl.**

```
#!/usr/bin/perl
print "Content-type: text/html\n\n";
print"<HTML><BODY><H1>Query Results</H1>\n";
print "You submitted the following Name/value pairs:\n";
if($ENV{'REQUEST_METHOD'}eq "POST"){
        $form=<STDIN>;
}else{
      $form=$ENV{'QUERY_STRING'};
}
@pairs=split(/&/,$form);
print "<UL>";
while(@pairs){
        $pair=shift@pairs;
                        $pair=~s/\+//g;
                        print "<LI><CODE>$pair</CODE>\n";
}
print "</UL>";
```

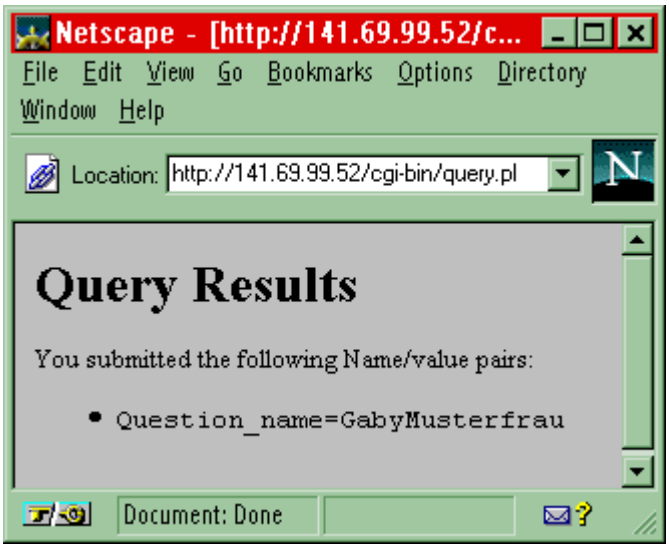
Das CGI- Programm gibt seine Daten über die stdout- Schnittstelle aus(Das Perl-Skript

verwendet zu diesem Zweck die print-Anweisung). Der WWW-Server leitet diese Daten dann an den Client weiter, der das CGI- Programm gestartet hat. Wichtig ist, daß das CGI-Programm den eigentlichen Ausgabedaten die folgenden Daten voranstellt:

- Definition des Inhalttyps : ***Content-type: text/html***
- Ausgabe einer Leerzeile : \n\n

Danach erzeugt das CGI- Programm durch Print- Anweisungen die Ausgabe der HTML-Seite.

3.) Der Server leitet die durch das CGI- Programm dynamisch generierte HTML-Seite an den Browser weiter. Dieser erzeugt im vorliegenden Fall die folgende Ausgabe:



Die Attribute POST / GET des <FORM>-Tag

Das obige Perl- Skript ermittelt zunächst durch Abfrage der Umgebungsvariablen 'REQUEST_METHOD' ob als Methode GET oder POST verwendet wurde.

Wird im <Form>-Tag das Attribut POST verwendet, erfolgt die Übergabe der Formulardaten an das CGI- Programm über die stdin- Schnittstelle.

Bei Verwendung des Attributes GET im <Form>-Tag, erfolgt die Kommunikation mit dem CGI-Programm über die Umgebungsvariable QUERY_STRING. Die Länge der übergebenen Zeichenkette ist bei Verwendung dieser Methode auf 255 Zeichen beschränkt.

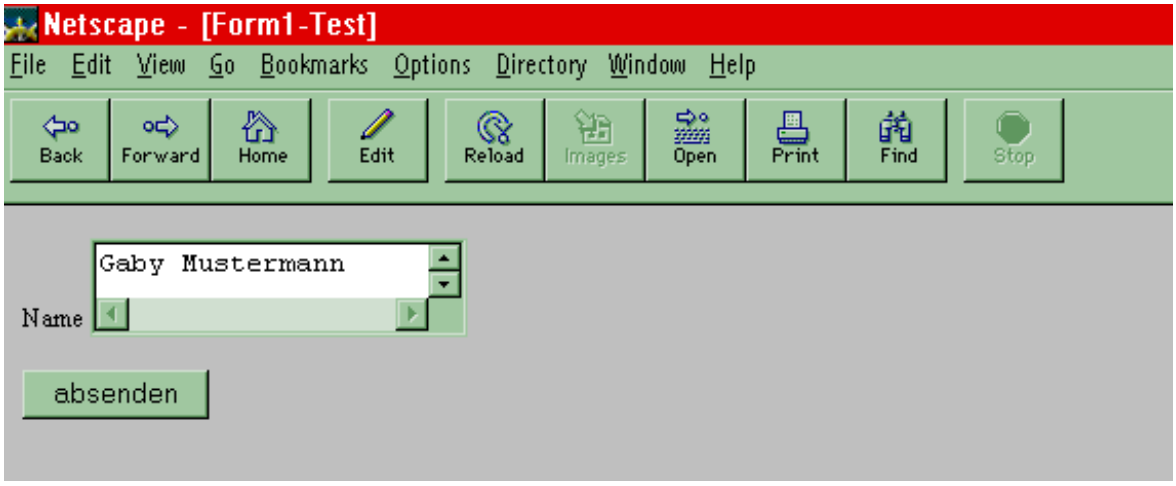
Bei Verwendung der Methode **GET** werden die Formulardaten an den URL angehängt. Sie sind dann also ein Bestandteil dieser URL. Da in einer URL keine Sonder- und Leerzeichen erlaubt sind, werden die Formulardaten in ein kodiertes Format umgewandelt. Das verwendete Format besteht aus einer einzigen Zeichenkette, die sich aus den Formularfeldnamen und Formularfeldwerten zusammensetzt. Name und Wert eines Formularfeldes werden durch ein Gleichheitszeichen(=) und jedes Name/Wert-Paar durch ein Kaufmanns-und (&) voneinander getrennt. Leerzeichen werden durch ein + dargestellt. Sonderzeichen werden als Prozentzeichen(%), gefolgt vom ASCII-Wert (in Hexadezimalnotation) des betreffenden Zeichens, dargestellt.

Die Datenrahmen, die bei Verwendung der Methode **POST**, ausgetauscht werden, sind im folgenden Abschnitt zusammengestellt.

1.) Ein Browser erhält folgende HTML-Seite zur Anzeige.

```
<HTML>
  <HEAD>
      <TITLE>Form1-Test</TITLE>
  </HEAD>
  <BODY>
      <FORM ACTION="http://141.69.99.52/cgi-bin/query.pl" METHOD="POST">
         Name <TEXTAREA NAME= "Question_name"></TEXTAREA><p>
              <INPUT TYPE="submit" VALUE="absenden">
     </FORM>
  </BODY>
</HTML>
```

2.) Im Browser erfolgt die Eingabe von Nutzerdaten in das Textfeld (Daten = Gaby Mustermann Franz Gurkenkönig)



3.) Durch das Anklicken des „absenden"-Buttons sendet der Client die folgenden Datenrahmen an den betreffenden Server.

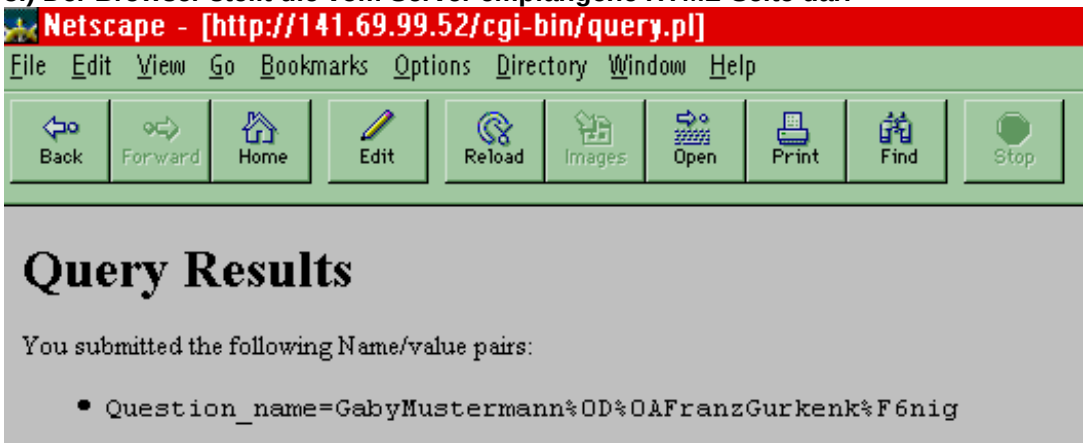
```
DUMP:                    ------ REST OF FRAME DATA ------
DUMP:
DUMP:   0000- 50 4F 53 54 20 2F 63 67  69 2D 62 69 6E 2F 71 75  POST /cgi-bin/qu
DUMP:   0010- 65 72 79 2E 70 6C 20 48  54 54 50 2F 31 2E 30 0D  ery.pl HTTP/1.0.
DUMP:   0020- 0A 52 65 66 65 72 65 72  3A 20 66 69 6C 65 3A 2F  .Referer: file:/
DUMP:   0030- 2F 2F 44 7C 2F 48 54 4D  4C 2F 43 67 69 2F 66 6F  //D|/HTML/Cgi/fo
DUMP:   0040- 72 6D 31 2E 68 74 6D 6C  0D 0A 43 6F 6E 6E 65 63  rm1.html..Connec
DUMP:   0050- 74 69 6F 6E 3A 20 4B 65  65 70 2D 41 6C 69 76 65  tion: Keep-Alive
DUMP:   0060- 0D 0A 55 73 65 72 2D 41  67 65 6E 74 3A 20 4D 6F  ..User-Agent: Mo
DUMP:   0070- 7A 69 6C 6C 61 2F 33 2E  30 47 6F 6C 64 20 28 57  zilla/3.0Gold (W
DUMP:   0080- 69 6E 39 35 3B 20 49 29  0D 0A 48 6F 73 74 3A 20  in95; I)..Host:
DUMP:   0090- 31 34 31 2E 36 39 2E 39  39 2E 35 32 0D 0A 41 63  141.69.99.52..Ac
DUMP:   00A0- 63 65 70 74 3A 20 69 6D  61 67 65 2F 67 69 66 2C  cept: image/gif,
DUMP:   00B0- 20 69 6D 61 67 65 2F 78  2D 78 62 69 74 6D 61 70   image/x-xbitmap
DUMP:   00C0- 2C 20 69 6D 61 67 65 2F  6A 70 65 67 2C 20 69 6D  , image/jpeg, im
DUMP:   00D0- 61 67 65 2F 70 6A 70 65  67 2C 20 2A 2F 2A 0D 0A  age/pjpeg, */*..

DUMP:                    ------ REST OF FRAME DATA ------
DUMP:
DUMP:   0000- 43 6F 6E 74 65 6E 74 2D  74 79 70 65 3A 20 61 70  Content-type: ap
DUMP:   0010- 70 6C 69 63 61 74 69 6F  6E 2F 78 2D 77 77 77 2D  plication/x-www-
DUMP:   0020- 66 6F 72 6D 2D 75 72 6C  65 6E 63 6F 64 65 64 0D  form-urlencoded.
DUMP:   0030- 0A 43 6F 6E 74 65 6E 74  2D 6C 65 6E 67 74 68 3A  .Content-length:
DUMP:   0040- 20 36 31 0D 0A 0D 0A 51  75 65 73 74 69 6F 6E 5F   61....Question_
```

```
DUMP:   0050- 6E 61 6D 65 3D 47 61 62  79 2B 4D 75 73 74 65 72  name=Gaby+Muster
DUMP:   0060- 6D 61 6E 6E 2B 25 30 44  25 30 41 46 72 61 6E 7A  mann+%0D%0AFranz
DUMP:   0070- 2B 47 75 72 6B 65 6E 6B  25 46 36 6E 69 67 25 30  +Gurkenk%F6nig%0
DUMP:   0080- 44 25 30 41 0D 0A                                 D%0A..
```

4.) Der Server leitet die Formulardaten an das im 1. Datenrahmen angegebene CGI-Programm weiter und startet es. Die Ausgabedaten des CGI-Programms werden vom Server aufgenommen und mit einem Header versehen an den Client zurückgesendet .

```
DUMP:                   ------ REST OF FRAME DATA ------
DUMP:
DUMP:   0000- 48 54 54 50 2F 31 2E 30  20 32 30 30 20 4F 4B 0D  HTTP/1.0 200 OK.
DUMP:   0010- 0A 44 61 74 65 3A 20 53  75 6E 2C 20 32 33 20 4D  .Date: Sun, 23 M
DUMP:   0020- 61 72 20 31 39 39 37 20  30 38 3A 35 32 3A 31 35  ar 1997 08:52:15
DUMP:   0030- 20 47 4D 54 0D 0A 53 65  72 76 65 72 3A 20 41 70   GMT..Server: Ap
DUMP:   0040- 61 63 68 65 2F 31 2E 30  2E 35 0D 0A 43 6F 6E 74  ache/1.0.5..Cont
DUMP:   0050- 65 6E 74 2D 74 79 70 65  3A 20 74 65 78 74 2F 68  ent-type: text/h
DUMP:   0060- 74 6D 6C 0D 0A 0D 0A 3C  48 54 4D 4C 3E 3C 42 4F  tml....<HTML><BO
DUMP:   0070- 44 59 3E 3C 48 31 3E 51  75 65 72 79 20 52 65 73  DY><H1>Query Res
DUMP:   0080- 75 6C 74 73 3C 2F 48 31  3E 0A 59 6F 75 20 73 75  ults</H1>.You su
DUMP:   0090- 62 6D 69 74 74 65 64 20  74 68 65 20 66 6F 6C 6C  bmitted the foll
DUMP:   00A0- 6F 77 69 6E 67 20 4E 61  6D 65 2F 76 61 6C 75 65  owing Name/value
DUMP:   00B0- 20 70 61 69 72 73 3A 0A  3C 55 4C 3E 3C 4C 49 3E   pairs:.<UL><LI>
DUMP:   00C0- 3C 43 4F 44 45 3E 51 75  65 73 74 69 6F 6E 5F 6E  <CODE>Question_n
DUMP:   00D0- 61 6D 65 3D 47 61 62 79  4D 75 73 74 65 72 6D 61  ame=GabyMusterma
DUMP:   00E0- 6E 6E 25 30 44 25 30 41  46 72 61 6E 7A 47 75 72  nn%0D%0AFranzGur
DUMP:   00F0- 6B 65 6E 6B 25 46 36 6E  69 67 25 30 44 25 30 41  kenk%F6nig%0D%0A
DUMP:   0100- 3C 2F 43 4F 44 45 3E 0A  3C 2F 55 4C 3E 0A 3C 2F  </CODE>.</UL>.</
DUMP:   0110- 42 4F 44 59 3E 3C 2F 48  54 4D 4C 3E              BODY></HTML>
```

5.) Der Browser stellt die vom Server empfangene HTML-Seite dar.



Aufruf eines Shell-Kommandos

Die Dateien müssen mit dem Dateiattribut ausführbar markiert werden.

Inhalt der Datei ps2 im Verzeichnis cgi-bin :

```
#!/bin/sh
echo Content-Type: text/html
echo
echo <html><body><pre>
ps -ef #Beispiel fuer ein UNIX-Kommando
echo </pre>></body></html>
```

Inhalt der Datei ls2 im Verzeichnis cgi-bin :

```
#!/bin/sh
echo Content-Type: text/html
echo
```

```
echo <html><body><pre>
ls -al #Beispiel fuer ein UNIX-Kommando
echo </pre></body></html>
```

CGI-Programm (GET - Methode)

```
/* cgi_get.c
 Dieses Programm wird vom CGI aufgerufen
 Es liest die Eingaben des zugehörigen Formulars über
 die Umgebungsvarable QUERY_STRING
 Das zugehörige Formular muß die Methode GET verwenden
*/
#include <stdio.h>
#include <stdlib.h>
void main(void){
        char c, *in,*http_user_agent,*query_string;
        int i=0;
        http_user_agent=getenv("HTTP_USER_AGENT");
        query_string=getenv("QUERY_STRING");
        printf("Content-type: text/html\n\n\n");
        if (http_user_agent!= NULL){
                printf("%s\n",http_user_agent);
        }
        else
          printf("HTTP_USER_AGENT nicht gesetzt\n");
        printf("<br><hr>Eingabe:  %s",query_string);
}
```

CGI-Programm getvars (GET und POST: Ausgabe der Parameter)

```
/*************************************************************************/
/**                                                                     **/
/**     hello.c-- standard "hello, world", to demonstrate basic CGI     **/
/**         programming, and use of the getcgivars() routine.           **/
/**                                                                     **/
/**     You can either paste the whole routine into your program, as I  **/
/**     did here, or you can create a separate object and header file.  **/
/**                                                                     **/
/*************************************************************************/

/***************** start of the getcgivars() module **********************/

/*************************************************************************/
/**                                                                     **/
/**     getcgivars.c-- routine to read CGI input variables into an      **/
/**         array of strings.                                           **/
/**                                                                     **/
/**     Written in 1996 by James Marshall, james@jmarshall.com, except  **/
/**     that the x2c() and unescape_url() routines were lifted directly **/
/**     from NCSA's sample program util.c, packaged with their HTTPD.   **/
/**                                                                     **/
/**     For the latest, see http://www.jmarshall.com/easy/cgi/ .        **/
/**                                                                     **/
/*************************************************************************/

#include <stdio.h>
#include <string.h>
#include <stdlib.h>

/** Convert a two-char hex string into the char it represents. **/
char x2c(char *what) {
   register char digit;

   digit = (what[0] >= 'A' ? ((what[0] & 0xdf) - 'A')+10 : (what[0] - '0'));
   digit *= 16;
   digit += (what[1] >= 'A' ? ((what[1] & 0xdf) - 'A')+10 : (what[1] - '0'));
   return(digit);
}

/** Reduce any %xx escape sequences to the characters they represent. **/
void unescape_url(char *url) {
    register int i,j;
```

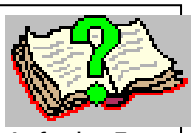
```
    for(i=0,j=0; url[j]; ++i,++j) {
        if((url[i] = url[j]) == '%') {
            url[i] = x2c(&url[j+1]) ;
            j+= 2 ;
        }
    }
    url[i] = '\0' ;
}


/** Read the CGI input and place all name/val pairs into list.        **/
/** Returns list containing name1, value1, name2, value2, ... , NULL  **/
char **getcgivars() {
    register int i ;
    char *request_method ;
    int content_length;
    char *cgiinput ;
    char **cgivars ;
    char **pairlist ;
    int paircount ;
    char *nvpair ;
    char *eqpos ;

    /** Depending on the request method, read all CGI input into cgiinput. **/
    request_method= getenv("REQUEST_METHOD") ;

    if (!strcmp(request_method, "GET") || !strcmp(request_method, "HEAD") ) {
        /* Some servers apparently don't provide QUERY_STRING if it's empty, */
        /*   so avoid strdup()'ing a NULL pointer here.                      */
        char *qs ;
        qs= getenv("QUERY_STRING") ;
        cgiinput= strdup(qs  ? qs  : "") ;
    }
    else if (!strcmp(request_method, "POST")) {
        /* strcasecmp() is not supported in Windows-- use strcmpi() instead */
        if ( strcasecmp(getenv("CONTENT_TYPE"), "application/x-www-form-urlencoded")) {
              printf("Content-Type: text/plain\n\n") ;
            printf("getcgivars(): Unsupported Content-Type.\n") ;
            exit(1) ;
        }
        if ( !(content_length = atoi(getenv("CONTENT_LENGTH"))) ) {
              printf("Content-Type: text/plain\n\n") ;
            printf("getcgivars(): No Content-Length was sent with the POST request.\n") ;
            exit(1) ;
        }
        if ( !(cgiinput= (char *) malloc(content_length+1)) ) {
              printf("Content-Type: text/plain\n\n") ;
            printf("getcgivars(): Couldn't malloc for cgiinput.\n") ;
            exit(1) ;
        }
        if (!fread(cgiinput, content_length, 1, stdin)) {
              printf("Content-Type: text/plain\n\n") ;
            printf("getcgivars(): Couldn't read CGI input from STDIN.\n") ;
            exit(1) ;
        }
        cgiinput[content_length]='\0' ;
    }
    else {
          printf("Content-Type: text/plain\n\n") ;
        printf("getcgivars(): Unsupported REQUEST_METHOD.\n") ;
        exit(1) ;
    }

    /** Change all plusses back to spaces. **/
    for (i=0; cgiinput[i]; i++) if (cgiinput[i] == '+') cgiinput[i] = ' ' ;

    /** First, split on "&" and ";" to extract the name-value pairs into **/
    /**   pairlist.                                                      **/
    pairlist= (char **) malloc(256*sizeof(char **)) ;
    paircount= 0 ;
    nvpair= strtok(cgiinput, "&;") ;
    while (nvpair) {
        pairlist[paircount++]= strdup(nvpair) ;
        if (!(paircount%256))
            pairlist= (char **) realloc(pairlist,(paircount+256)*sizeof(char **)) ;
        nvpair= strtok(NULL, "&;") ;
    }
    pairlist[paircount]= 0 ;    /* terminate the list with NULL */

    /** Then, from the list of pairs, extract the names and values. **/
    cgivars= (char **) malloc((paircount*2+1)*sizeof(char **)) ;
```

```
    for (i= 0; i<paircount; i++) {
        if (eqpos=strchr(pairlist[i], '=')) {
            *eqpos= '\0' ;
            unescape_url(cgivars[i*2+1]= strdup(eqpos+1)) ;
        } else {
            unescape_url(cgivars[i*2+1]= strdup("")) ;
        }
        unescape_url(cgivars[i*2]= strdup(pairlist[i])) ;
    }
    cgivars[paircount*2]= 0 ;   /* terminate the list with NULL */

    /** Free anything that needs to be freed. **/
    free(cgiinput) ;
    for (i=0; pairlist[i]; i++) free(pairlist[i]) ;
    free(pairlist) ;

    /** Return the list of name-value strings. **/
    return cgivars ;

}
/***************** end of the getcgivars() module ********************/
/** Standard "hello, world" program, that also shows all CGI input. **/
int main() {
    char **cgivars ;
    int i ;

    /** First, get the CGI variables into a list of strings         **/
    cgivars= getcgivars() ;

    /** Print the CGI response header, required for all HTML output. **/
    /** Note the extra \n, to send the blank line.                  **/
    printf("Content-type: text/html\n\n") ;

    /** Finally, print out the complete HTML response page.         **/
    printf("<html>\n") ;
    printf("<head><title>CGI Results</title></head>\n") ;
    printf("<body>\n") ;
    printf("<h1>Hello, world.</h1>\n") ;
    printf("Your CGI input variables were:\n") ;
    printf("<ul>\n") ;

    /** Print the CGI variables sent by the user.  Note the list of **/
    /**   variables alternates names and values, and ends in NULL.  **/
    for (i=0; cgivars[i]; i+= 2)
        printf("<li>[%s] = [%s]\n", cgivars[i], cgivars[i+1]) ;

    printf("</ul>\n") ;
    printf("</body>\n") ;
    printf("</html>\n") ;

    /** Free anything that needs to be freed **/
    for (i=0; cgivars[i]; i++) free(cgivars[i]) ;
    free(cgivars) ;
    exit(0) ;
}
```

Übersicht der CGI-Umgebungsvariablen

Siehe: SELFHTML http://selfhtml.org

Variablenname	Erläuterung
CONTENT_LENGTH	Enthält die Anzahl der Zeichen, die beim Aufruf des CGI-Scripts über die POST-Methode übergeben wurden. Wenn das CGI-Script beispielsweise beim Absenden eines HTML-Formulars aufgerufen wurde und dort als Übertragungsmethode POST angegeben ist, steht in dieser Umgebungsvariablen, wie viele Zeichen das Skript von der Standardeingabe lesen muß, um die übermittelten Formulardaten vollständig einzulesen.
CONTENT_TYPE	Enthält beim Aufruf über die POST-Methode den Mime-Type der übergebenen Daten. Wenn das CGI-Script beispielsweise beim Absenden eines HTML-Formulars aufgerufen wurde und dort als Übertragungsmethode POST angegeben ist, steht in dieser Umgebungsvariablen der für HTML-Formulare typische Mime-Type *application/x-www-form-urlencoded*.
GATEWAY_INTERFACE	Enthält die Version der CGI-Schnittstelle, die von dem installierten Server unterstützt wird, z.B. CGI/1.1, wenn die gegenwärtig übliche Version 1.1 der Schnittstellendefinition unterstützt wird.
HTTP_ACCEPT	Enthält die Liste der Mime-Types, die der aufrufende WWW-Browser akzeptiert. Die Angabe */* bedeutet: der WWW-Browser akzeptiert alles.
HTTP_REFERER	Enthält die URL-Adresse der WWW-Seite, von der aus das CGI-Script aufgerufen wurde. Der Wert wird jedoch nicht von allen WWW-Browsern übermittelt, ist also nicht in jedem Fall verfügbar.
HTTP_USER_AGENT	Enthält Produkt- und Versionsinformationen zum aufrufenden WWW-Browser. Ein CGI-Script kann auf diese Weise ermitteln, welchen Browser ein Anwender verwendet.
PATH_INFO	Enthält spezielle Pfadinformationen, wenn das Script über die GET-Methode aufgerufen wurde, und zwar relativ zu den im WWW-Server eingestellten Root-Verzeichnissen.
PATH_TRANSLATED	Enthält spezielle Pfadinformationen, wenn das Script über die GET-Methode aufgerufen wurde - im Unterschied zu PATH_INFO jedoch Pfadangaben entsprechend der Verzeichnisstruktur des Server-Rechners.
QUERY_STRING	Enthält beim Aufruf über die GET-Methode eine Zeichenkette mit Daten, die dem Script übergeben wurden. Wenn ein Anwender ein HTML-Formular ausgefüllt hat, bei dessen Absenden das CGI-Script aufgerufen wurde, stehen in dieser Umgebungsvariablen die ausgefüllten Formulardaten. Die Daten sind nach den Regeln des Mime-Types *application/x-www-form-urlencoded* kodiert.
REMOTE_ADDR	Enthält die IP-Adresse des Server-Rechners, über den das CGI-Script aufgerufen wurde, wenn es ein anderer Server ist als der, auf dem das CGI-Script liegt. Dieser Wert wird jedoch nicht in allen Fällen gesetzt.
REMOTE_HOST	Enthält die Domain-Adresse des Server-Rechners, über den das CGI-Script aufgerufen wurde, wenn es ein anderer Server ist als der, auf dem das CGI-Script liegt. Dieser Wert wird jedoch nicht in allen Fällen gesetzt.
REMOTE_IDENT	Enthält Protokollinformation, wenn auf dem Server das Protokoll *ident* für geschützte Zugriffe läuft.
REMOTE_USER	Enthält den Benutzernamen, der das CGI-Script aufgerufen hat. Dieser Wert wird nur gesetzt, wenn die Server-Authentifizierung eingeschaltet ist.
REQUEST_METHOD	Enthält den HTTP-Befehl, mit dem das CGI-Programm aufgerufen wurde. Entweder GET oder POST. Ein CGI-Script kann beispielsweise diese Variable auslesen und danach entscheiden, wie es Formulardaten einlesen kann: entweder von der Standardeingabe (bei Methode POST) oder aus der Umgebungsvariablen QUERY_STRING (bei Methode GET).
SCRIPT_NAME	Enthält die URL-Adresse des aufgerufenen CGI-Scripts realtiv zur Domain-Adresse, also z.B. /cgi-bin/search.pl.
SERVER_NAME	Enthält den Namen des Servers (z.B. www.xy.com) oder die IP-Adresse (z.B. 127.0.0.1).
SERVER_PORT	Enthält die Portnummer, die für den WWW-Server eingerichtet wurde. Normalerweise ist dies für WWW-Server die Nummer 80.
SERVER_PROTOCOL	Enthält die Version des HTTP-Protokolls, das der installierte WWW-Server unterstützt, z.B. HTTP/1.0, wenn die gegenwärtig übliche Version 1.0 des HTTP-Protokolls unterstützt wird.
SERVER_SOFTWARE	Enthält die Produktbezeichnung der installierten Server-Software, z.B. OmniHTTPd/2.0a1 (Win32; i386)



Aufgabe 5

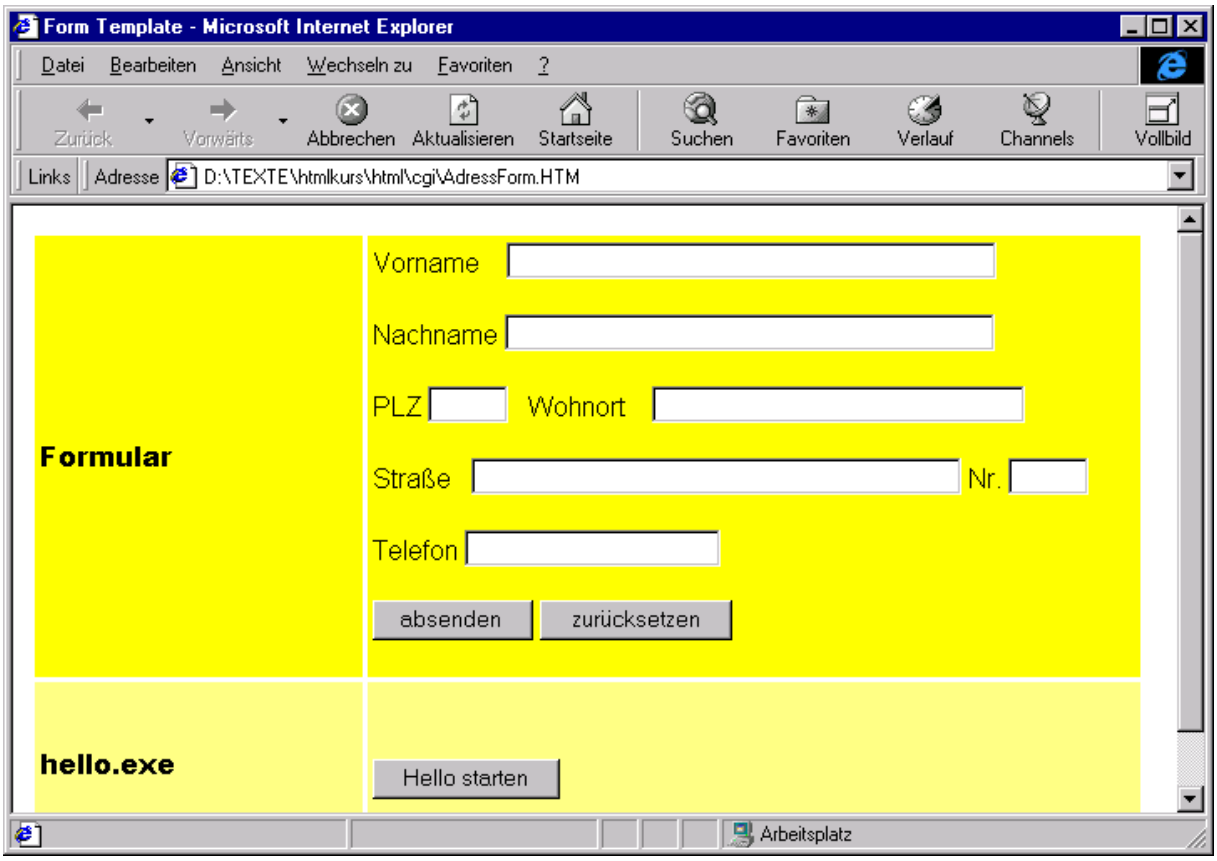
1.) Welche Methode verwendet das Formular, dessen Daten im obigen http-Trace gesendet werden? Wie wird bei dieser Methode der Wert der Eingabefelder an das CGI übergeben? An welches CGI-Script wird die Anfrage gesendet?

2.) Wie werden nicht druckbare Zeichen an das CGI gesendet?

3.) Wie identifiziert sich der Netscape-Navigator 3.0Gold?

4.) Wie identifiziert sich der Server?

5.)Erstellen Sie eine HTML-Datei die das untenstehende Formular erzeugt.

a.) Erstellen Sie ein CGI-Programm, welches durch das Betätigen des Buttons Hello starten aufgerufen wird und an den Browser Hello World zurückgibt.

b.)Verwenden Sie das Programm hello.c (Listing siehe oben) um die Formulardaten auszugeben.